GRID 2 — the wind, current, wave and depth grids

The format the router reads, and the format the course's converters write. `docs/web/route/grid.js` is the reference: **what it refuses, this format does not allow**, and where this document and that file differ, the file is right.

Why it exists on its own

Two converters write it — GRIB to wind, the Shom atlas to current — and the router reads it. The weather app may read it later. A format defined inside its first consumer becomes whatever that consumer happens to do, so it is defined here, with a reader that is strict on purpose.

Strict on purpose, because of what a bad grid becomes. It does not become an error; it becomes a *route* — a boat sailing through a gap in the wind, or an hour of stream applied to the wrong hour. A grid that is wrong in a way nothing notices is worse than one that will not load, because the router will draw a perfectly convincing track from it.

Two versions, and both are read. GRID 1 is the surface wind and the surface current, and is described first because everything in it holds for GRID 2 as well. GRID 2 adds waves, wind at a level other than the deck, and a depth, which has no time axis at all. A GRID 1 file means exactly what it always meant.

GRID 1

The shape

Plain text. Blank lines and lines beginning # are ignored anywhere. Values are separated by spaces or tabs. `\r\n` is read the same as `\n`.

```
GRID 1
kind wind
units kt
u east
v north
lat 49.00 51.00 0.25
lon -6.00 -1.00 0.25
times 2025-05-24T00:00Z 2025-05-24T03:00Z 2025-05-24T06:00Z
fields u v gust
order time lat lon
rows 2187
12.4 -3.1 18.0
12.1 -3.4 17.6
- - -
…
```

The header

Line	Value	Notes
`GRID 1`	—	Must be the first line. The version is the whole line.
`kind`	`wind` or `current`	
`units`	`kt`	Only knots. See below.
`u`	`east`	Positive east. Stated in the file, not only here.
`v`	`north`	Positive north.
`lat`	from to step	Increasing, step > 0, the span a whole number of steps.
`lon`	from to step	The same. No wrap across ±180 in GRID 1.
`times`	one or more	`YYYY-MM-DDTHH:MMZ`, UTC, strictly increasing.
`fields`	`u v` then extras	`gust` may follow on a `wind` grid.
`order`	`time lat lon`	The only order GRID 1 has.
`rows`	a whole number	**Last line of the header.**

Each line appears **once**. Any other key is refused rather than ignored: a header line nothing reads is a line somebody believes.

The data

One row per cell, `fields` values each, in the order `order` gives: time slowest, then latitude, then longitude fastest. A value of `-` means **no data here**, which is not the same as zero and does not read back as zero.

The three things worth explaining

One unit, one sign convention, one order

A format that accepts knots and metres per second is a format where half the files are in the other one and nothing says which until a boat is doing forty knots in a gale. The same goes for sign and for index order. Each is fixed, each is written in the file, and the reader refuses anything else. Half the bugs in this class of code are a sign, and a convention that lives only in a document is one that gets assumed.

The row count is checked twice, against two independent things

1. `rows` against `times × lat × lon`, derived from the grid description.
2. `rows` against the number of data rows actually in the file.

The first catches a header that disagrees with itself. **The second catches a truncated file**, which is the failure that actually happens to a text file: a copy that stopped, a pipe that closed, a disk that filled. It is one number in the header and one comparison in the reader, and it is the same reasoning the NT1 check code is built on — the realistic fault is mechanical, not malicious.

Spans must be a whole number of steps

`(to − from) / step` is floating point. Quarter-degree steps over two degrees must give 8, not 7.999999999999999, so the count is rounded and then checked against the rounding. A span that is not a whole number of steps is refused rather than silently truncated, because a silently truncated grid is a grid with a missing edge and no sign of one.

Reading one

```
var g = GRID.parse(text);
if (!g.ok) { /* g.err is one sentence; g.line is where */ }
g.meta.times.length           // how many times
g.at(ti, yi, xi)              // { u: …, v: …, gust: … }, null values where "-"
g.lat(yi)  g.lon(xi)          // the position of an index
```

It never throws. A bad grid is a result with one plain sentence and a line number, because a reader that throws in a router is a route that disappears rather than a grid that is wrong.

What neither version does

1. **No interpolation.** Reading a grid and sampling it between cells are different jobs; the second belongs to the router.
2. **No wrap at ±180.** The Channel does not need it. When something does, it is the next version, and old files still read.
3. **No projection and no ensembles.**

GRID 2

GRID 2 is the same grid, for three more things the course asks of it: **waves**, **wind at a level other than the deck**, and a **depth**, which has no time axis at all. `grid.js` reads both versions, and a GRID 1 file means exactly what it meant.

```
GRID 2
kind wind
level surface
method model
units kt kt kt
u east
v north
lat 49.00 51.00 0.25
lon -6.00 -1.00 0.25
times 2025-05-24T00:00Z 2025-05-24T03:00Z
fields u v gust
order time lat lon
rows 1458
12.4 -3.1 18.0
…
```

Why they are here rather than in a format of their own

practical-navigator asked whether the course's four selectable overlays belonged in this format at all. They do, and the reason that decided it is **not** that they are convenient to put here:

1. **Two of the four are the router's**, not only the course's. A sea state becomes a speed penalty on the polar; a depth becomes a minimum depth under the keel. Anything the router feeds on belongs in the format the router reads.
2. **A 925 hPa wind loaded as the surface wind is the exact fault this format exists to prevent.** It would not error. It would route — on a wind perhaps a quarter to a third too strong and **veered** from the real surface wind, because friction backs the surface wind out of the gradient wind, so loading the gradient wind as the surface one puts that rotation back the wrong way. The only defence is that the level is **in the file**, and that the router refuses a grid whose level is not `surface` — which is only possible if upper winds live in this format rather than beside it.
3. **They are the same mechanical object:** values on a regular latitude–longitude grid. What is expensive here is not the wind — it is the span that must be a whole number of steps, the row count checked twice, `-` meaning no data, and a reader that never throws. A second format gets all of that wrong again, or copies it and drifts.

The level, and how the field was arrived at

Line	Value	
`level`	`surface`, or a pressure level such as `925hPa`	**Required on a wind grid.** Never defaulted
`method`	`model` or `geostrophic`	**Required on a wind grid.** How the field was arrived at

They are two lines because they are two different facts: a geostrophic wind is worked out from the isobars *at a level*, and somebody will one day want one at 925 hPa. The three winds a lesson compares are then three files that differ by two lines: `surface`/`model`, `925hPa`/`model`, and `surface`/`geostrophic`.

A defaulted level is a convention living in a document, which is the thing this format refuses to do with units and with signs. A GRID 1 wind needs no level: GRID 1 *is* the surface format, and says so in its own version line.

`COURSE.checkGrids()` refuses a wind grid whose level is not `surface`, by name, because the boat sails in the surface wind. Upper winds are for reading.

The unit belongs to the field

`units kt` cannot carry metres, seconds and degrees, so in GRID 2 **`units` is one unit per field, in the order `fields` gives**, and each is checked against a fixed dictionary — a file may not say `ft`:

```
kind wave
units  m  s  degT  m      s      degT
fields hs tp dirto hs_sea tp_sea dirto_sea
```

field	unit	
`u` `v` `gust`	`kt`	as GRID 1
`hs`	`m`	significant height
`tp`	`s`	peak period
`dirto`	`degT`	**the direction the sea runs TOWARDS**, like a current's set and the opposite of how a wind is named
`hs_sea` `tp_sea` `dirto_sea`		the wind-sea partition
`hs_swell` `tp_swell` `dirto_swell`		the swell partition
`z`	`m`	depth, **positive down from chart datum** — the same datum as the tide heights, so a depth and a height can be added. A drying height is negative

`dirto` carries its convention in its own name, so nothing downstream has to remember which way round it is, and the converter does the flip once, where the source's convention is known and can be checked. It is the single most likely place for a wave layer to be silently wrong.

The `units` line is a copy of what the field names already fix. It is kept because a file somebody opens has to be readable on its own, and it is checked so that the copy cannot drift — which is the rule the project applies everywhere else to a copy that must exist.

A grid with no time axis

```
kind depth
units  m
fields z
times none
order lat lon
rows 129
```

1. **`times none` is written, not omitted.** An omitted line cannot be told from a truncated header, and the header's fixed shape is this format's whole defence against a file that stopped halfway.
2. **`order lat lon` follows from it,** and `rows` is then latitudes × longitudes. Both count checks still run, both ways.
3. **It has one slice, not none:** `at(0, yi, xi)` reads it, and `meta.timeless` is true.

What a GRID 2 grid is refused for

Everything GRID 1 refuses, and:

1. a wind grid with no `level` or no `method`, or either of those on a grid that is not a wind;
2. `u` or `v` on a wave or depth grid — those lines are a vector's sign convention, and a line nothing reads is a line somebody believes;
3. `units` that does not give exactly one unit per field, or gives a unit the field is not in;
4. `times none` with an order that still mentions time, or times with an order that does not;
5. a kind's required fields missing, or a field that kind does not carry.

Changing it

A new field on an existing kind is not a version change: `fields` says what each row carries, and a reader that does not know an extra field still reads the ones it needs. Anything else — a unit, a sign, an order, a new header line that must be understood — is the next version, and `grid.js` keeps reading the ones before it.